

Ôn tập: 1. Tổng quan về máy học — Lời giải

Họ tên: ... Mã SV: ...

1.1. Cơ bản

Câu 1. Máy học (Machine Learning) là gì? Phân biệt máy học với lập trình truyền thống.

Trả lời. Máy học là lĩnh vực nghiên cứu và xây dựng các phương pháp cho phép máy tính rút ra quy luật hoặc quyết định hữu ích từ *dữ liệu* thay vì do con người lập trình từng quy tắc cố định cho mọi tình huống.

So với lập trình truyền thống, điểm khác cốt lõi nằm ở chỗ **ai** cung cấp logic xử lý:

- **Lập trình truyền thống:** con người viết rõ quy tắc đầu vào–đầu ra; máy chỉ thực thi đúng những gì được mô tả.
- **Máy học:** con người thường chỉ cung cấp dữ liệu (và đôi khi nhãn) cùng một họ các mô hình và thuật toán tối ưu; máy *ước lượng* quy luật từ dữ liệu.

Khi bài toán phức tạp, khó viết quy tắc tay (nhận diện giọng nói, hình ảnh), máy học thường phù hợp hơn; khi quy tắc nghiệp vụ rõ ràng và ít thay đổi, lập trình truyền thống có thể đơn giản và tin cậy hơn.

Câu 2. Một bài toán máy học thường gồm những thành phần nào (dữ liệu, mô hình, hàm mất mát, thuật toán học)?

Trả lời. Thành phần thường gặp gồm:

- **Dữ liệu:** tập mẫu mô tả đối tượng cần học (đặc trưng, nhãn nếu có).
- **Mô hình (model):** họ các hàm ánh xạ từ đặc trưng sang dự đoán, có tham số cần học.
- **Hàm mất mát (loss):** đo mức sai lệch giữa dự đoán và mục tiêu; định hướng cần cải thiện khi huấn luyện.
- **Thuật toán học (learning algorithm):** quy trình cập nhật tham số (ví dụ hạ gradient) nhằm giảm hàm mất mát (hoặc tối đa hóa hàm mục tiêu tương đương).

Ngoài ra trong thực tế còn cần **đánh giá** trên tập kiểm tra, **tiền xử lý** dữ liệu, và lựa chọn siêu tham số.

Câu 3. Thế nào là mô hình (model) trong máy học?

Trả lời. Mô hình là khung toán học mô tả mối quan hệ giữa đặc trưng đầu vào và đầu ra cần dự đoán. Mô hình có **tham số** (học được từ dữ liệu) và có thể có **siêu tham số** do người thiết kế chọn trước. Ví dụ: mô hình tuyến tính, cây quyết định, mạng nơ-ron đều là các họ mô hình khác nhau.

Câu 4. Dữ liệu huấn luyện (training data) và dữ liệu kiểm tra (test data) khác nhau như thế nào?

Trả lời.

- **Huấn luyện:** dùng để ước lượng tham số mô hình (học quy luật).
- **Kiểm tra: tách biệt**, không tham gia huấn luyện; dùng để đánh giá khả năng **khái quát** (generalization) lên dữ liệu mới.

Nếu đánh giá trên cùng dữ liệu đã học, kết quả thường **lạc quan giả tạo** vì mô hình có thể “nhớ” nhiều hoặc chi tiết riêng của tập huấn luyện.

Câu 5. Nhãn (label) là gì? Phân biệt bài toán có nhãn và không nhãn.

Trả lời. Nhãn là giá trị mục tiêu đã biết cho từng mẫu (ví dụ: “spam/không spam”, nhiệt độ dự báo, mã lỗi bệnh).

Phân biệt:

- **Có nhãn (supervised):** mỗi mẫu có cặp (x, y) ; mục tiêu là học ánh xạ $x \mapsto y$ (phân lớp hoặc hồi quy).
- **Không nhãn (unsupervised):** chỉ có x ; mục tiêu thường là tìm cấu trúc (cụm, giảm chiều, mật độ, luật kết hợp, ...).

1.2. Các loại bài toán máy học

Câu 1. Phân biệt học có giám sát, học không giám sát và học tăng cường.

Trả lời.

- **Học có giám sát:** dữ liệu gồm nhãn; mô hình học ánh xạ từ đặc trưng đến nhãn. Kiểu đầu ra: nhãn rời rạc (phân lớp) hoặc liên tục (hồi quy).
- **Học không giám sát:** dữ liệu không có nhãn; tập trung vào cấu trúc nội tại (phân cụm, giảm chiều, mô hình sinh, phát hiện bất thường theo mật độ, ...).
- **Học tăng cường:** tác tử tương tác với **môi trường** qua hành động; nhận tín hiệu thưởng/phạt; mục tiêu học **chính sách** tối đa hóa phần thưởng tích lũy theo thời gian. Điểm khác có giám sát là tín hiệu thưởng **thưa, trễ** và phụ thuộc chuỗi hành động.

Câu 2. Cho ví dụ thực tế cho từng loại bài toán máy học.

Trả lời.

- **Có giám sát:** phát hiện gian lận thẻ khi có nhãn giao dịch “gian lận/tốt”; dự báo mưa theo ảnh vệ tinh có đo thực địa.
- **Không giám sát:** phân nhóm khách hàng theo lịch sử mua hàng khi chưa gán nhãn nhóm; giảm chiều dữ liệu gene để trực quan hóa.
- **Tăng cường:** huấn luyện robot đi bộ trong mô phỏng; chơi cờ với phần thưởng thắng/thua; điều khiển hệ thống tiết kiệm năng lượng với chi phí chạy máy.

Câu 3. Phân biệt bài toán phân lớp (classification) và hồi quy (regression).

Trả lời. Cả hai đều thuộc học có giám sát, nhưng **dạng nhãn** khác nhau:

- **Phân lớp:** nhãn rời rạc (lớp); mô hình dự đoán lớp hoặc phân phối xác suất trên các lớp.
- **Hồi quy:** nhãn (mục tiêu) là số thực (hoặc vector số); mô hình dự đoán giá trị số (ví dụ giá nhà, nồng độ).

Ngưỡng hóa đầu ra hồi quy thành khoảng rời rạc đôi khi được dùng trong thực tế, nhưng về bản chất bài toán vẫn được xếp theo loại nhãn gốc.

Câu 4. Phân biệt clustering và classification.

Trả lời.

- **Phân lớp:** học có giám sát; **có nhãn lớp** cho tập huấn luyện; mục tiêu gán lớp cho mẫu mới theo nhãn hiện có.
- **Phân cụm (clustering):** học không giám sát; **không có nhãn**; mục tiêu tìm nhóm mẫu tương đồng (cụm) dựa trên khoảng cách hoặc mật độ.

Kết quả cụm không nhất thiết trùng với “lớp nghiệp vụ”; cần diễn giải hoặc đối chiếu thêm với chuyên môn.

Câu 5. Bài toán dự đoán giá nhà thuộc loại bài toán nào? Vì sao?

Trả lời. Đây là bài toán **hồi quy trong học có giám sát**. Mục tiêu là dự đoán **giá nhà**—một biến liên tục (hoặc rất nhiều mức giá khác nhau)—từ các đặc trưng đầu vào (diện tích, vị trí, ...), khi đã có lịch sử các cặp đặc trưng–giá thực tế để huấn luyện.

1.3. Mô hình và thuật toán cơ bản

Câu 1. Perceptron là gì? Perceptron có những hạn chế nào?

Trả lời. Perceptron (đơn thần kinh, kết hợp hàm kích hoạt bước) là mô hình phân lớp **nhị phân tuyến tính**: nó học một **siêu phẳng** tách hai lớp bằng cập nhật trọng số khi gặp mẫu phân loại sai.

Hạn chế chính:

- Chỉ tách được dữ liệu **tuyến tính tách được** (linearly separable); với dữ liệu không tách tuyến tính, thuật toán cổ điển có thể không hội tụ ổn định.
- Không trực tiếp cho **xác suất lớp**; biên quyết định “cứng”.
- Bài toán đa lớp cần mở rộng (one-vs-rest, ...).

Câu 2. Hồi quy tuyến tính (Linear Regression) là gì?

Trả lời. Hồi quy tuyến tính mô hình hóa mục tiêu y như tổ hợp **tuyến tính** của đặc trưng (có thể kèm hệ số chặn): $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$. Tham số thường được ước lượng bằng cách **tối thiểu hóa sai số bình phương trung bình** (bình phương độ lệch giữa $\hat{y}$ và y trên tập huấn luyện). Đây là mô hình **cơ sở** cho nhiều phương pháp hồi quy phức tạp hơn (ridge, lasso, mạng nơ-ron với đầu ra tuyến tính).

Câu 3. K-means là gì? Thuật toán k-means hoạt động như thế nào?

Trả lời. K-means là thuật toán **phân cụm** chia n mẫu thành k cố định sao cho tổng bình phương khoảng cách trong cụm tới **tâm cụm** nhỏ.

Cách làm lặp (Lloyd):

1. Khởi tạo k tâm (ngẫu nhiên hoặc k-means++).
2. **Gán cụm:** mỗi điểm gán vào tâm gần nhất.
3. **Cập nhật tâm:** tâm mới là trung bình các điểm trong cụm.
4. Lặp đến khi không đổi cụm (hoặc đủ vòng lặp).

Kết quả phụ thuộc khởi tạo và giá trị k ; hàm mục tiêu là **lỗi cục bộ** nên nhiều lần chạy khác nhau có thể cho cụm khác nhau.

Câu 4. Cây quyết định (Decision Tree) là gì? Ưu và nhược điểm của cây quyết định.

Trả lời. Cây quyết định là mô hình dự đoán biểu diễn bằng **cây**: mỗi nút trong kiểm tra một điều kiện trên đặc trưng, nhánh dẫn tới nút con, lá gán nhãn hoặc giá trị dự đoán.

Ưu điểm:

- Dễ diễn giải trực quan; chọn lọc đặc trưng tự nhiên qua cấu trúc cây.
- Ít cần chuẩn hóa đặc trưng (so với nhiều mô hình tuyến tính / khoảng cách).
- Xử lý được đặc trưng hỗn hợp (số, phân loại) tùy cách triển khai.

Nhược điểm:

- Dễ **overfitting** nếu cây quá sâu.
- Biên quyết định trục-song-song hạn chế (cây thường xây bằng cắt song song với trục)—khó mô hình hóa biên phức tạp đơn cây nông.
- Ổn định kém so với **tập hợp** cây (ví dụ rừng) nếu dữ liệu nhiễu nhẹ cũng đổi cấu trúc.

Câu 5. Naïve Bayes là gì? Giả định “naïve” trong Naïve Bayes là gì?

Trả lời. Naïve Bayes là bộ phân lớp dựa trên định lý Bayes, ước lượng xác suất lớp hậu nghiệm $P(y | \mathbf{x})$ thông qua $P(\mathbf{x} | y)$ và $P(y)$.

Giả định “naïve” là các đặc trưng **độc lập có điều kiện** khi đã biết lớp:

$$P(\mathbf{x} | y) = \prod_j P(x_j | y).$$

Giả định này thường **không đúng** trong thực tế nhưng giúp ước lượng với ít tham số và tính nhanh; vẫn hiệu quả ở nhiều bài toán văn bản (đặc trưng túi từ).

1.4. Huấn luyện mô hình và tối ưu

Câu 1. Hàm mất mát (loss function) là gì? Vai trò của hàm loss trong quá trình học.

Trả lời. Hàm mất mát đo mức “sai” của mô hình trên dữ liệu (một mẫu hoặc cả tập), thường là hàm của tham số mô hình. Ví dụ: bình phương sai số cho hồi quy, log-loss cho phân lớp.

Vai trò: định nghĩa **mục tiêu tối ưu** trong huấn luyện—thuật toán học cố gắng tìm tham số làm **giảm** tổng mất mát (hoặc kỳ vọng mất mát), từ đó kéo dự đoán gần nhãn hoặc mục tiêu hơn.

Câu 2. Gradient Descent là gì? Phân biệt batch, mini-batch và stochastic gradient descent.

Trả lời. Gradient Descent là phương pháp tối ưu lặp: cập nhật tham số theo hướng **ngược gradient** của hàm mục tiêu (mất mát) để giảm giá trị hàm.

Phân biệt theo **kích thước mẫu** dùng để tính gradient mỗi bước:

- **Batch GD:** dùng toàn bộ tập huấn luyện mỗi bước; ước lượng gradient ổn định nhưng tốn chi phí khi dữ liệu lớn.
- **Mini-batch GD:** dùng một **lô nhỏ** mẫu; cân bằng giữa ổn định và tốc độ, phổ biến trong huấn luyện sâu.
- **SGD (thuần):** mỗi bước dùng **một** mẫu (hoặc vài mẫu); nhiễu gradient lớn nhưng có thể thoát vùng lồi nhạt, cập nhật rất nhanh trên tập lớn.

Câu 3. Learning rate là gì? Điều gì xảy ra khi learning rate quá lớn hoặc quá nhỏ?

Trả lời. **Learning rate** (tốc độ học) là hệ số nhân cho bước cập nhật theo hướng gradient. Nó điều khiển **độ lớn** mỗi bước di chuyển trong không gian tham số.

- **Quá lớn:** bước nhảy mạnh, dễ **dao động** quanh điểm tối ưu hoặc **phát tán** (loss tăng, không hội tụ).
- **Quá nhỏ:** hội tụ chậm, dễ kẹt ở vùng **gradient nhỏ**, tốn thời gian huấn luyện.

Câu 4. Overfitting và underfitting là gì?

Trả lời.

- **Underfitting:** mô hình **quá đơn giản** so với dữ liệu; không nắm được quy luật cốt lõi, nên sai cả trên tập huấn luyện và tập kiểm tra.
- **Overfitting:** mô hình **quá phức tạp** hoặc học quá kỹ; khớp nhiều và chi tiết riêng của tập huấn luyện, khiến **lỗi trên tập kiểm tra** tụt dù lỗi huấn luyện nhỏ.

Câu 5. Regularization là gì? Vì sao cần regularization?

Trả lời. **Regularization** là kỹ thuật thêm **độ phạt** vào hàm mục tiêu nhằm hạn chế mô hình quá phức tạp (thường phạt độ lớn trọng số, ví dụ L_2 ridge, L_1 lasso).

Lý do cần thiết: giúp cải thiện **khái quát**, giảm overfitting, ổn định nghiệm khi dữ liệu nhiễu hoặc đa cộng tuyến; đôi khi đồng thời chọn lọc đặc trưng (lasso).

1.5. Đánh giá mô hình

Câu 1. Accuracy là gì? Khi nào accuracy không phù hợp để đánh giá mô hình?

Trả lời. **Accuracy** (độ chính xác tổng thể) là tỷ lệ dự đoán đúng trên tổng số mẫu.

Accuracy **kém tin cậy** khi:

- Dữ liệu **mất cân bằng lớp** (một lớp chiếm đa số lớn)—mô hình “luôn đoán lớp đa số” vẫn accuracy cao.
- Chi phí **sai khác nhau giữa các kiểu lỗi** (ví dụ bỏ sót bệnh nghiêm trọng đắt hơn báo động giả).
- Cần biết chất lượng **từng lớp** thay vì chỉ tỷ lệ đúng chung.

Câu 2. Ma trận nhầm lẫn (confusion matrix) là gì?

Trả lời. Ma trận nhầm lẫn là bảng đếm (hoặc tỷ lệ) so sánh **nhân thực** và **nhân dự đoán** trong phân lớp. Với phân lớp nhị phân, các ô thường gồm **true positive**, **true negative**, **false positive**, **false negative**. Ma trận cho thấy mô hình nhầm theo **hướng nào** (ghi nhận nhầm lớp dương, bỏ sót lớp dương, ...).

Câu 3. Precision, Recall, F1-score là gì? Khi nào nên ưu tiên Recall hơn Precision?

Trả lời. Với lớp dương quan tâm:

- **Precision** (độ chính xác dương): trong số mẫu d báo dương, bao nhiêu thật sự dương. Đo “tin cậy khi báo có”.
- **Recall** (độ phủ, sensitivity): trong số mẫu thật dương, bao nhiêu được tìm ra. Đo “khả năng không bỏ sót dương”.
- **F1-score:** trung bình điều hòa của precision và recall, cân bằng hai mặt.

Ưu tiên recall hơn khi chi phí **bỏ sót dương** rất cao, ví dụ sàng lọc bệnh nguy hiểm, phát hiện xâm nhập: ta chấp nhận nhiều cảnh báo nhầm hơn là bỏ qua tình huống dương thật.

Câu 4. Phân biệt Sensitivity và Specificity.

Trả lời. Trong phân lớp nhị phân (thường “có tình trạng” vs “không”):

- **Sensitivity (độ nhạy):** tỷ lệ **thật dương** được nhận ra đúng—trùng **recall** của lớp dương ($TP/(TP+FN)$).
- **Specificity (độ đặc hiệu):** tỷ lệ **thật âm** được nhận ra đúng ($TN/(TN+FP)$). Do khả năng tránh báo động sai trên lớp âm.

Hai chỉ số bổ sung cho nhau khi cần cân nhắc **cả hai loại sai lệch**.

Câu 5. ROC curve và AUC là gì?

Trả lời. Đường cong ROC là đồ thị mô tả đổi ngược giữa **tỷ lệ dương thật** (độ nhạy, recall) và **tỷ lệ dương giả** trên lớp âm thật, khi ta thay đổi **ngưỡng** phân lớp của mô hình cho điểm xếp hạng.

AUC (area under ROC) là diện tích dưới đường ROC; tóm tắt hiệu năng trên nhiều ngưỡng. AUC gần 1 nghĩa là mô hình có khả năng **phân hạng** tốt (dương thường có điểm cao hơn âm); $AUC \approx 0,5$ gần như ngẫu nhiên. AUC có thể lạc quan khi mất cân bằng lớp nặng; khi đó cần thêm **PR curve / PR-AUC**.

1.6. Dữ liệu và tiền xử lý

Câu 1. Tiền xử lý dữ liệu (data preprocessing) là gì?

Trả lời. Tiền xử lý là các bước biến đổi dữ liệu thô trước khi huấn luyện hoặc suy luận nhằm: làm sạch, chuẩn hóa thang đo, mã hóa biến phân loại, xử lý giá trị ngoại lai, giảm nhiễu, và đôi khi tạo đặc trưng mới. Mục tiêu là giúp thuật toán học **ổn định, công bằng giữa đặc trưng, và hiệu quả** hơn.

Câu 2. Xử lý dữ liệu thiếu (missing data) như thế nào?

Trả lời. Phụ thuộc **cơ chế thiếu** (MCAR/MAR/MNAR) và tỉ lệ thiếu. Các hướng phổ biến:

- **Loại bỏ** mẫu hoặc đặc trưng thiếu nhiều nếu còn đủ dữ liệu.
- **Điền giá trị:** dùng trung bình, median, mode, hoặc ước lượng thiếu bằng mô hình trên tập huấn luyện.
- **Mã hóa cờ “thiếu”** như một tín hiệu bổ sung khi việc thiếu mang thông tin.

Quan trọng là **chỉ ước lượng** quy tắc điền/thay thế trên tập huấn luyện, rồi **áp dụng cùng quy tắc** cho tập kiểm tra, tránh để thông tin từ tập kiểm tra lọt ngược vào bước tiền xử lý.

Câu 3. Chuẩn hóa (normalization) và chuẩn hóa Z-score khác nhau như thế nào?

Trả lời.

- **Min–max normalization:** đưa giá trị về khoảng cố định (thường $[0, 1]$) theo min/max trên tập huấn luyện. Phụ thuộc **những đuôi** và khoảng biến thiên.

- **Z-score (standardization):** trừ trung bình, chia độ lệch chuẩn: đưa về thang có trung bình ≈ 0 , phương sai ≈ 1 . Ít phụ thuộc min/max cực trị hơn so với min-max nhưng vẫn nhạy outlier nếu không xử lý.

Cả hai đều giúp các thuật toán dựa trên khoảng cách hoặc gradient không bị một đặc trưng quy mô lớn át đặc trưng khác.

Câu 4. Tại sao cần chia dữ liệu thành training / validation / test?

Trả lời.

- **Huấn luyện:** học tham số mô hình.
- **Kiểm định (validation):** chọn siêu tham số, so sánh biến thể mô hình, dừng sớm—**không** dùng để báo cáo khả năng khái quát cuối cùng nếu đã lặp nhiều lần “nhìn” vào nó.
- **Kiểm tra (test):** đánh giá một lần (hoặc rất hạn chế) khi đã khóa pipeline; ước lượng hiệu năng thực tế trên dữ liệu chưa tham gia huấn luyện/chọn mô hình.

Chia tách giúp **giảm đánh giá quá lạc quan** do overfitting lên cùng dữ liệu.

Câu 5. Feature engineering là gì?

Trả lời. Feature engineering là tạo, biến đổi, kết hợp đặc trưng để biểu diễn bài toán rõ ràng hơn với mô hình (ví dụ log-transform biến lệch, tạo tỷ lệ giữa hai biến, mã hóa thứ tự có ý nghĩa, trích bit ngày giờ). Đây thường là bước **tác động lớn** tới hiệu năng trong thực tế.

1.7. So sánh và nhận xét

Câu. So sánh Perceptron và Logistic Regression.

Trả lời. Cả hai đều là mô hình **tuyến tính** quyết định biên phân chia dựa trên tổ hợp tuyến tính $w^T x + b$, thường cho **phân lớp nhị phân**.

Điểm khác chủ yếu:

- **Hàm mất mát và đầu ra:** Perceptron cổ điển dùng ngưỡng 0/1 và quy tắc cập nhật sai lầm; logistic regression dùng hàm sigmoid cho **xác suất** $P(y=1 | x)$ và thường tối ưu **log-loss** (hợp lý cực đại).
- **Xác suất:** logistic regression trực tiếp cho điểm xác suất, tiện ngưỡng hóa và hiệu chỉnh xác suất; perceptron cổ điển không.
- **Khả năng tối ưu:** logistic regression với log-loss là bài toán lồi mượt hơn (thuận lợi cho gradient descent); perceptron phụ thuộc khả năng tách tuyến tính trong biến thể cổ điển.

Câu hỏi bổ sung (đứng trước mục 1.8 trong đề ôn tập)

Câu 1. So sánh k-means và hierarchical clustering.

Trả lời.

- **Bản chất:** k-means phân hoạch dữ liệu thành k **cố định** cụm, tối thiểu hóa tổng bình phương trong cụm; phân cụm phân cấp xây **cây (dendrogram)** gom dần hoặc tách dần cụm, thường không cần chọn k ngay từ đầu.

- **Dạng biên cụm:** k-means giả định cụm gần “**hình cầu**” quanh tâm; phân cấp có thể phù hợp hơn cụm **không lồi** trong một số phương án liên kết.
- **Chi phí và quy mô:** k-means thường nhanh trên dữ liệu lớn (đặc biệt mini-batch); nhiều thuật toán phân cấp đầy đủ có độ phức tạp cao hơn, khó mở rộng tập rất lớn.
- **Ổn định:** k-means nhạy khởi tạo; phân cấp một lần cho cấu trúc đầy đủ, nhưng thứ tự hoặc hàm liên kết cũng ảnh hưởng.

Câu 2. So sánh Decision Tree và Random Forest.

Trả lời.

- **Cấu trúc:** một cây quyết định đơn là mô hình duy nhất, dễ diễn giải; **random forest** là **tập hợp** nhiều cây, mỗi cây huấn luyện trên **bootstrap** và/hoặc **chọn ngẫu nhiên tập con đặc trưng** tại mỗi nút, dự đoán cuối là bỏ phiếu hoặc trung bình.
- **Khái quát hóa:** rừng giảm overfitting và phương sai so với một cây sâu, thường **ổn định và chính xác** hơn trên dữ liệu thực; đổi lại kém diễn giải hơn.
- **Dữ liệu:** cả hai xử lý tốt đặc trưng hỗn hợp nếu triển khai hỗ trợ; rừng cần nhiều tài nguyên hơn (lưu trữ và suy luận nhiều cây).

Câu 3. Khi nào nên dùng mô hình đơn giản thay vì mô hình phức tạp?

Trả lời. Nên ưu tiên mô hình đơn giản khi:

- Dữ liệu **ít** hoặc nhiều, hai lớp gần tách tuyến tính—mô hình phức tạp dễ overfit.
- Cần **giải thích**, kiểm toán, hoặc triển khai nhẹ (độ trễ thấp, bộ nhớ nhỏ).
- Điểm cơ sở (baseline) phải rõ ràng để so sánh xứng đáng với mô hình lớn.
- Chi phí huấn luyện và bảo trì cao so với lợi ích dự kiến từ độ phức tạp.

Nguyên tắc thực hành là **bắt đầu đơn giản, chỉ phức tạp hóa khi có bằng chứng lợi ích** trên kiểm định/test hợp lệ.

Câu 4. Vì sao không tồn tại mô hình “tốt nhất cho mọi bài toán”?

Trả lời. Hiệu năng phụ thuộc **bài toán, dữ liệu, chi phí sai, và ràng buộc triển khai**. Không có định lý “phổ quát tối ưu”: một mô hình mạnh ở dữ liệu có cấu trúc tuyến tính có thể yếu với dữ liệu phi tuyến sâu; thuật toán tốn nhiều mẫu có thể kém khi dữ liệu hiếm. Hơn nữa, **định lý “No Free Lunch”** trong tối ưu hóa học máy cho rằng, trên mọi bài toán có thể mô tả một cách trung lập, không có thuật toán nào luôn vượt trội mọi thuật toán khác. Vì vậy cần **chọn và thử nghiệm** theo ngữ cảnh.

1.8. Các vấn đề khác

Câu 1. Cho tập dữ liệu các cặp (x, y) với y chỉ nhận hai giá trị Có/Không. Nhận định loại bài toán máy học và nêu mô hình hoặc thuật toán phù hợp.

Trả lời. Đây là bài toán **học có giám sát, phân lớp nhị phân**: mỗi mẫu có nhãn thuộc một trong hai lớp.

Các hướng phù hợp (ví dụ): **hồi quy logistic, cây quyết định / rừng ngẫu nhiên, Naïve Bayes, máy vector hỗ trợ (SVM)**, hoặc mạng nơ-ron nhỏ cho dữ liệu phi tuyến. Lựa chọn cụ thể phụ thuộc kích thước dữ liệu, khả năng tách lớp, nhu cầu xác suất đầu ra và khả năng diễn giải.

Câu 2. Hệ thống dự đoán giá nhà từ diện tích, số phòng, vị trí. Nhận định loại bài toán và dạng đầu ra của mô hình.

Trả lời. Đây là **học có giám sát, hồi quy**: mục tiêu là dự đoán **giá**—biến số thực (hoặc đếm tiền làm tròn liên tục) từ đặc trưng đầu vào.

Dạng đầu ra: một số thực $\hat{y}$ (giá dự báo) hoặc khoảng tin cậy nếu mô hình ước lượng thêm phân phối sai số.

Câu 3. Dữ liệu giao dịch ngân hàng **không có nhãn**, mục tiêu phát hiện nhóm khách hàng có hành vi tương tự. Nhận định bài toán và một thuật toán phù hợp.

Trả lời. Đây là **học không giám sát**, cụ thể là **phân cụm** (clustering): tìm cấu trúc nhóm trong dữ liệu khi chưa biết nhãn nhóm.

Thuật toán gợi ý: **k-means** (khi có ý niệm khoảng cách Euclidean trên đặc trưng đã chuẩn hóa), **DBSCAN** hoặc **tối ưu Gaussian mixture** khi cụm có hình dạng/kích thước khác nhau, hoặc **phân cụm phân cấp** khi cần biểu diễn cấu trúc đa mức—tùy giả định về dữ liệu và khối lượng tính toán.

Câu 4. Hệ thống học chơi cờ bằng tương tác môi trường, nhận thưởng/phạt sau mỗi hành động. Nhận định loại bài toán và đặc điểm cách học.

Trả lời. Đây là **học tăng cường (reinforcement learning)**: tác tử chọn hành động, quan sát phản hồi môi trường và tín hiệu phần thưởng.

Đặc điểm chính: tín hiệu huấn luyện là **thưởng/trễ** (không phải nhãn đúng cho từng trạng thái ngay lập tức); mục tiêu tối ưu **phần thưởng tích lũy theo chính sách**; có **dò tìm–khai thác**; dữ liệu là **chuỗi tương tác** nên không độc lập như nhiều bài có giám sát cổ điển.

Câu 5. Tập dữ liệu lớn: lớp bình thường 95%, lớp bất thường 5%. Nhận định vấn đề chính và đề xuất hướng đánh giá mô hình phù hợp.

Trả lời. Vấn đề chính là **mất cân bằng lớp nghiêm trọng**: nếu chỉ tối ưu accuracy, mô hình có thể **phớt lờ lớp hiếm** (bất thường) nhưng vẫn đạt tỷ lệ đúng cao trên tập chung.

Hướng đánh giá phù hợp:

- Ưu tiên các chỉ số theo lớp: **precision, recall, F1** của lớp bất thường; hoặc **Balanced accuracy, MCC**.
- Dùng **confusion matrix** để thấy rõ số lượng false negative (bỏ sót bất thường).
- Xem **đường PR và PR-AUC** (thường nhạy hơn ROC khi lớp hiếm).
- Chia tập **stratified** hoặc giữ tỷ lệ lớp ở validation/test; cân nhắc chi phí nghiệp vụ để chọn ngưỡng (ưu tiên recall nếu bỏ sót rất đắt).